

Small Language Model Challenge

Parameter Matched SwiGLU with Capacity Scaling and Dropout

Course	Student	Student ID	Date
DASE7506	Li Zhoutong	3036781272	29 September 2026

1 Summary

I reproduced the supplied causal language-model baseline before changing the model. My first experiment replaced only the GELU feed-forward sublayer with a parameter-matched SwiGLU gate. With the same 1,200 updates and 9,830,400 processed targets, validation BPB decreased from 2.071083 to 2.010852. I then increased model capacity and added dropout. The frozen final model reached 1.567424 validation BPB and 1.589531 CPU FP32 test BPB, which is 24.35% lower than the reproduced baseline test score.

The report separates three questions. The matched experiment tests SwiGLU itself. A second matched-capacity experiment tests dropout at 7,200 updates. The 12,000-update model is the submitted quality-cost choice; it is not used to claim that SwiGLU alone produced the whole final improvement.

Final test BPB	Baseline test BPB	CPU scoring	Peak RAM	Assets
1.589531	2.101260	42.99 s (3.19x)	1.4972 GiB	20.46 MiB

1.1 Task and evaluation rule

The course asks for a causal next-token predictor trained only on the supplied training split, selected on validation and scored on test with the fixed evaluator [1]. The data are the supplied WikiText-2 raw splits, introduced with the WikiText benchmark by Merity et al. [2]. The BPE vocabulary has 2,048 tokens and every evaluation window contains at most 256 targets. Windows are independent, so the model cannot carry state from one window to the next.

BPB = total negative log2 probability of scored tokens / raw UTF-8 byte count

BPB is the ranking metric and lower is better. I kept the tokenizer, data files, causal window construction and evaluator unchanged.

2 Baseline model and improvement choices

The baseline is a decoder-only Transformer in the general architecture introduced by Vaswani et al. [3]. It contains four pre-LayerNorm blocks, model width 128 and four attention heads. Each head therefore has dimension 32. Learned token and absolute-position embeddings are added before the blocks, and the token embedding matrix is tied to the output classifier. The baseline has 1,088,256 trainable parameters.

Component	Supplied baseline	Possible change	Decision in this project
Token and position input	BPE-2048 plus learned absolute positions	Rotary or relative position encodings	Kept fixed because context is only 256
Self-attention	4-head causal scaled dot-product attention	Width, heads or a new attention rule	Kept the same rule; scaled width and heads together
Feed-forward	GELU MLP with hidden width 512	Gating or different hidden allocation	Replaced with parameter-matched SwiGLU
Residual and norm	Pre-LayerNorm residual blocks	RMSNorm or changed residual layout	Kept fixed for a cleaner mechanism test
Regularisation	No dropout	Embedding and residual-branch dropout	Added dropout 0.1 after overfitting appeared
Training budget	1,200 updates	Longer training and larger capacity	Increased only after validation experiments

2.1 Baseline reproduction

I ran the unmodified baseline with seed 17, batch size 32, context 256, four CPU threads and FP32. Training took 769.99 seconds on an AMD Ryzen 7 4800HS. Validation BPB was 2.071083 and CPU FP32 test BPB was 2.101260, matching the staff reference of approximately 2.10. The checkpoint fingerprint begins 7d869d and ends 6d3f.

2.2 Why I did not replace the attention mechanism

The starter model already implements multi-head masked self-attention through scaled_dot_product_attention with is_causal=True. Therefore, changing it to 'multi-head masked attention' would only rename the existing mechanism. A new attention design was possible, but it would have required additional correctness checks and a separate ablation. I chose the feed-forward sublayer because it could be changed without altering causality, positions or the evaluator.

2.3 Why the feed-forward sublayer was a practical target

The feed-forward network is applied independently to every token after attention and contains a large share of each block's matrix weights. It can change how features are selected while leaving the sequence interaction mechanism untouched. This made it possible to compare GELU and SwiGLU at almost exactly the same total parameter count.

3 Model and training modifications

3.1 Parameter-matched SwiGLU

SwiGLU is a gated linear-unit variant studied for Transformer feed-forward layers by Shazeer [4]. The baseline sends one projected feature path through GELU. My version learns a gate and a value path, multiplies them elementwise, and projects the result back to the model width.

```
GELU MLP(x) = W2 GELU(W1 x + b1) + b2
SwiGLU(x) = W0 [SiLU(Wg x + bg) * (Wv x + bv)] + b0
```

A SwiGLU layer has three main matrices rather than two. Keeping hidden width 512 would therefore make the student model much larger. I used hidden width 341 for the controlled run. Its total parameter count is 1,088,424, only 168 parameters or 0.015% above baseline. Width, depth, attention, LayerNorm, positions, initialization, optimiser, sampling, seed and processed targets remain fixed. The GELU baseline is consequently the removal ablation for the gate.

3.2 Capacity scaling

After the matched run improved validation, I increased model width from 128 to 256 and depth from four to six blocks. I increased the number of attention heads from four to eight so that head dimension stayed at 32 rather than changing two attention properties at once. The SwiGLU hidden width becomes 688, close to the same parameter-matching rule used in the small model. The final network has 5,355,584 parameters, 4.92 times the baseline count.

3.3 Dropout regularisation

The larger model without dropout reached its best recorded intermediate validation BPB at 3,600 updates and then became worse by 7,200 updates. I interpreted this pattern as overfitting. I added dropout 0.1, following the general regularisation method of Srivastava et al. [5]. In `student_model.py` it is applied to the summed token-position embedding, the attention probabilities during training, the projected attention residual branch and the SwiGLU residual branch. PyTorch automatically disables these dropout operations during evaluation, so CPU scoring is deterministic for a fixed checkpoint.

3.4 Longer training

The final run uses 12,000 updates and batch size 32, so it processes 98,304,000 targets. I retained the supplied AdamW settings: initial learning rate 0.001, weight decay 0.1, 100-update warm-up, cosine decay to one tenth of the peak rate, and gradient-norm clipping at 1.0. Training used the local NVIDIA GPU in FP32 to reduce wall-clock time. Model selection and the reported ranked score still use the fixed validation and CPU FP32 evaluation rules.

4 Experimental design and recorded search cost

4.1 Main runs

Run	Width / heads / blocks	Dropout	Updates	Targets	Parameters	Purpose
GELU baseline	128 / 4 / 4	0	1,200	9,830,400	1,088,256	Reference
Matched SwiGLU	128 / 4 / 4	0	1,200	9,830,400	1,088,424	Gate ablation
Wider SwiGLU	192 / 6 / 4	0	3,600	29,491,200	2,223,232	Earlier frozen candidate
Large SwiGLU	256 / 8 / 6	0	7,200	58,982,400	5,355,584	No-dropout control
Large + dropout	256 / 8 / 6	0.1	7,200	58,982,400	5,355,584	Dropout ablation
Final large + dropout	256 / 8 / 6	0.1	12,000	98,304,000	5,355,584	Frozen submission

4.2 What each comparison can establish

- GELU baseline versus matched SwiGLU isolates the gating change because parameters and target count are nearly identical.
- The two 256-wide, six-block 7,200-update runs isolate dropout because configuration, seed, target count and parameter count match.
- The final model changes capacity, dropout and training duration relative to baseline. It measures the final quality-cost trade-off, not a single causal factor.

4.3 Development and test policy

I used validation BPB for architecture and checkpoint decisions. The matched SwiGLU and the two 7,200-update dropout runs were validation experiments and were not scored on test. Test BPB is reported for the reproduced baseline, an earlier frozen 192-wide milestone and the current frozen submission. The current model was selected at 1.567424 validation BPB before its test evaluation.

4.4 Training and search cost

Run group	Hardware	Recorded training time
Baseline and matched SwiGLU	4-thread Ryzen 7 4800HS CPU	769.99 s + 724.22 s
192-wide candidate	4-thread Ryzen 7 4800HS CPU	3,483.08 s
Three 256-wide development/final runs	GTX 1660 Ti Max-Q GPU, FP32	1,931.05 s + 1,619.97 s + 2,207.84 s
Six main runs total	Mixed CPU and GPU; times not directly comparable	10,736.16 s (2.98 h wall-clock sum)

Short smoke tests added less than one minute of training. The total above discloses completed main runs; it is not a claim about equivalent compute because the CPU and GPU differ.

5 Experimental results

Method	Validation BPB	Test BPB	Training time	Selection status
GELU baseline	2.071083	2.101260	769.99 s CPU	Reference
Matched SwiGLU	2.010852	Not evaluated	724.22 s CPU	Validation-only ablation
Wider SwiGLU 192	1.710366	1.738670	3,483.08 s CPU	Earlier frozen milestone
Large SwiGLU, no dropout	1.706230	Not evaluated	1,931.05 s GPU	Validation-only control
Large SwiGLU, dropout 0.1	1.592719	Not evaluated	1,619.97 s GPU	Validation-only ablation
Final dropout model	1.567424	1.589531	2,207.84 s GPU	Current frozen submission

Controlled 1,200-update comparison (seed 17)

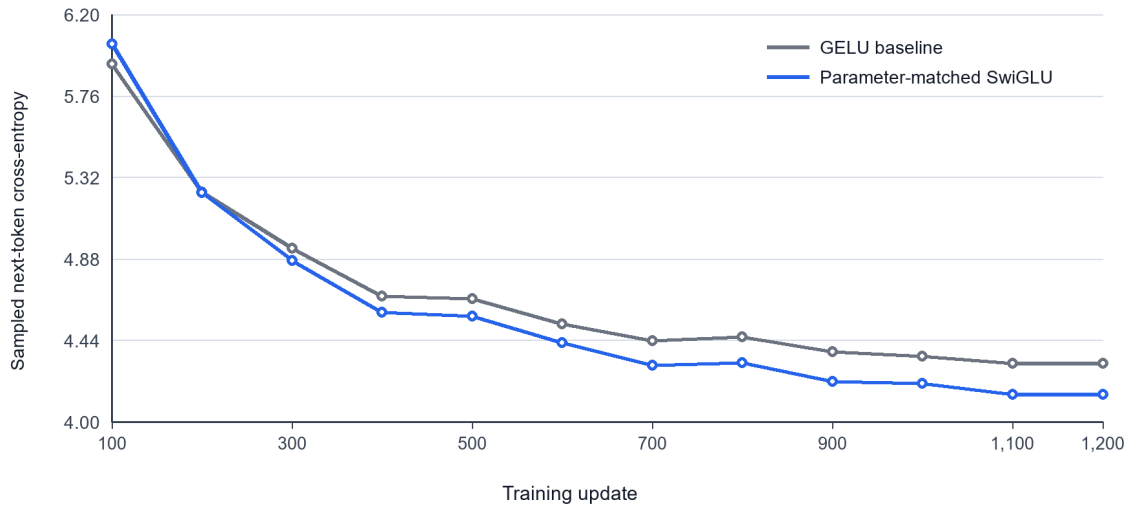


Figure 1. Sampled training loss for the parameter-matched mechanism comparison. Values are read directly from the two metrics.json histories; validation BPB, not sampled batch loss, determines selection.

5.1 Reading Figure 1

The first 100-update observation favours GELU, but the curves are nearly equal by update 200. From update 300 through update 1,200, every logged SwiGLU loss is lower. The final validation difference is 0.060232 BPB, a 2.91% relative reduction. Because the models differ by only 168 parameters and see the same targets, additional capacity does not explain this comparison.

The matched run trained slightly faster on this laptop, but I do not treat one local timing as evidence that SwiGLU is inherently faster. Background processes and kernel timing were not controlled through repeated trials.

6 Dropout and final model analysis

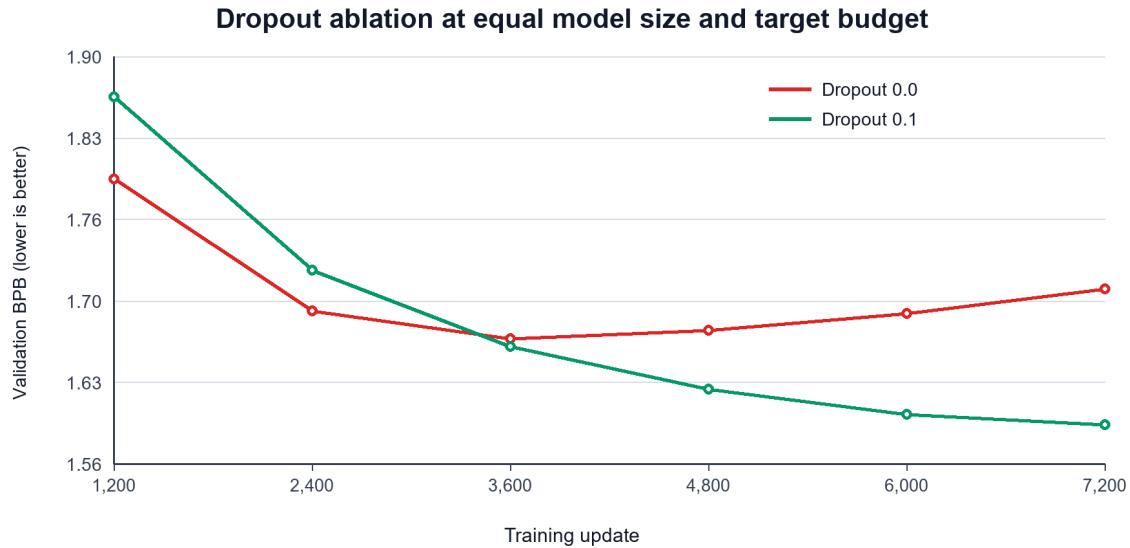


Figure 2. Validation BPB for equal 5.36M-parameter, 7,200-update runs. Points are loaded from `validation_history` in the saved `metrics.json` files. Test data are not used in this comparison.

6.1 Dropout ablation

Without dropout, validation BPB improves to 1.664660 at update 3,600, then degrades to 1.706230 at update 7,200. With dropout 0.1, validation BPB continues to improve at every recorded checkpoint and finishes at 1.592719. At the equal 7,200-update endpoint, dropout reduces validation BPB by 0.113510, or 6.65% relative. This is evidence for the regularisation choice at the large-model scale.

6.2 Effect of additional capacity and training

The 192-wide model shows that increasing capacity and the training budget can lower BPB substantially: validation falls from 2.010852 for matched SwiGLU to 1.710366. The 256-wide model has more capacity again, but the no-dropout curve shows that capacity alone is not sufficient; validation begins to worsen with continued training. Adding dropout makes the longer schedule useful.

6.3 Final 12,000-update result

The final model uses the same width, depth, head count, hidden width and dropout rate as the 7,200-update dropout run. Its validation BPB is 1.567424, another 0.025296 below the shorter run. The two jobs were separate GPU runs, so I treat this as supporting evidence for the longer schedule rather than an exact checkpoint continuation. After freezing this model, CPU FP32 test evaluation produced 1.589531 BPB.

6.4 Overall improvement

Relative to the reproduced baseline, the final model reduces validation BPB by 0.503660 (24.32%) and test BPB by 0.511730 (24.35%). The cost is 4.92 times as many parameters and 10 times as many processed training targets. The gain therefore combines the validated SwiGLU gate, more capacity, dropout and a longer schedule.

7 Evaluation cost and reproducibility

7.1 Resource limits

Constraint	Course limit	Baseline	Final model	Status
CPU FP32 scoring time	At most 5x baseline	13.46 s	42.99 s (3.19x)	Pass
Peak evaluation RAM	At most 4 GiB	1.5384 GiB	1.4972 GiB	Pass
Uncompressed inference assets	At most 64 MiB	4.171 MiB	20.46 MiB	Pass

The time ratio uses measurements on the same Ryzen laptop with four threads. Peak RAM is the process-tree working set from a repeated evaluation. Inference assets count the checkpoint, student factory, student model implementation and configuration. The final checkpoint is therefore within every stated evaluation limit.

7.2 Checkpoint identity

SHA-256 is a file fingerprint rather than a model score. Any byte change produces a different value, so the evaluator can verify that a downloaded checkpoint matches the one used for the report. Tables shorten fingerprints only for readability. The complete current checkpoint SHA-256 is:

```
90163aed08c92f1affce1830907e63a62ef0b01ec32327c82a71088f8c747802
```

7.3 Reproduction commands

From the repository root, install the pinned CPU environment and run the contract tests:

```
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

Reproduce the controlled SwiGLU run:

```
python train.py --implementation student --config configs/swiglu_128.json --device cpu --precision fp32
--threads 4 --seed 17 --steps 1200 --batch-size 32 --run-dir runs/swiglu128-fair-s17
```

Reproduce final training on CUDA, or omit `--device cuda` to train more slowly on CPU, then run ranked CPU evaluation:

```
python train.py --implementation student --config configs/swiglu_256x6_dropout.json --device cuda
--precision fp32 --threads 4 --seed 17 --steps 12000 --batch-size 32 --eval-every 2000 --run-dir
runs/swiglu256x6-dropout10-final12000-s17
python evaluate.py --checkpoint runs/swiglu256x6-dropout10-final12000-s17/checkpoint.pt --device cpu
--precision fp32 --threads 4 --split test
```

8 Limitations and conclusion

8.1 Limits of the evidence

- Each complete comparison uses seed 17 once. I therefore cannot estimate variance across random initialisations.
- The matched SwiGLU result isolates the gate only at the 1.09M-parameter scale; it does not prove the same percentage gain at every model size.
- The dropout ablation is controlled at 5.36M parameters and 7,200 updates, but the final 12,000-update job is a separate GPU run rather than a continuation of the 7,200-update checkpoint.
- CPU and GPU training times are reported for transparency but cannot be compared as algorithmic speed measurements.
- The final improvement cannot be assigned to a single change because width, depth, dropout and training duration all differ from baseline.

8.2 Conclusion

The controlled experiment supports replacing the baseline GELU MLP with parameter-matched SwiGLU: validation BPB improves by 2.91% with essentially the same parameter and target budgets. The large-model ablation then shows why dropout matters. Without dropout, validation worsens after 3,600 updates; with dropout 0.1, it continues to improve through 7,200 updates and is 6.65% lower at the matched endpoint. Scaling the selected design to six blocks and training the regularised model for 12,000 updates produces the final 1.589531 CPU FP32 test BPB while remaining under the time, memory and asset limits.

References

- [1] DASE7506. MP1 Small Language Model Challenge starter package and evaluation protocol. 2026.
- [2] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer Sentinel Mixture Models. arXiv:1609.07843, 2016. <https://arxiv.org/abs/1609.07843>
- [3] A. Vaswani et al. Attention Is All You Need. Advances in Neural Information Processing Systems 30, 2017. <https://arxiv.org/abs/1706.03762>
- [4] N. Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020. <https://arxiv.org/abs/2002.05202>
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research 15, 2014, pp. 1929-1958.